ANNA UNIVERSITY, CHENNAI-600 025

COLLEGE OF ENGINEERING GUINDY

A CNN BASED DEEP LEARNING MODEL FOR DETECTING P2P BOTNETS USING FLOW FEATURES

A PROJECT REPORT

Submitted by

A T PRANAV VARSHAN 2025165034

in partial fulfilment for the award of the degree

of

MASTER OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

(DATA SCIENCE AND CYBER SECURITY)

OBJECTIVES:

The objective of this project is to analyze network traffic and detect botnet activities using important flow-based features. A Convolutional Neural Network (CNN) model is developed to identify P2P botnet attacks and classify network traffic as normal or malicious. The performance of the model is evaluated using standard metrics such as accuracy, precision, recall, and F1-score.

ABSTRACT:

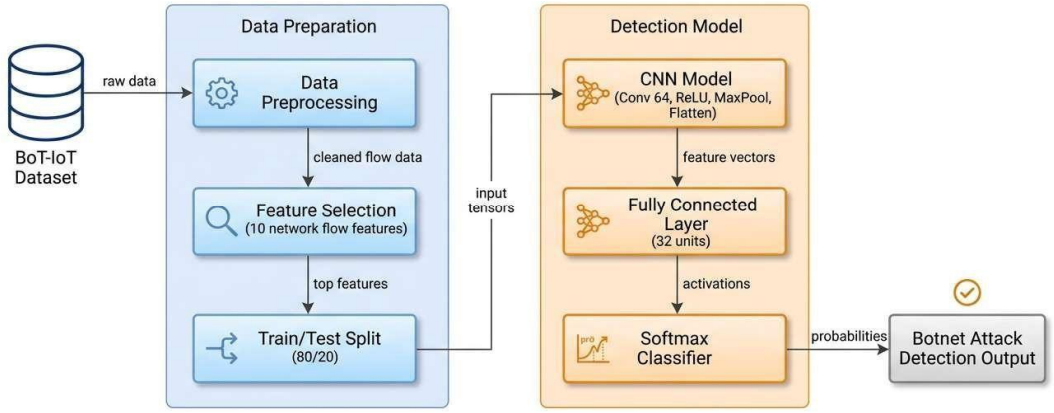
Botnets are a major cybersecurity threat in modern networks, especially with the rapid growth of Internet-connected devices. Among them, Peer-toPeer (P2P) botnets are difficult to detect due to their decentralized structure, which eliminates reliance on a central command server and reduces the effectiveness of traditional detection methods.This project proposes a deep learning-based approach for detecting P2P botnet activities using network traffic flow features. A Convolutional Neural Network (CNN) model is used to automatically learn patterns from a processed subset of the BoT-IoT dataset (UNSW IoT Botnet dataset). The dataset is preprocessed through cleaning, encoding, and feature selection before being split into training and testing sets. The CNN model, consisting of convolutional, pooling, and fully connected layers, classifies network traffic into normal or botnet activity. Experimental results show that the model can effectively detect botnet traffic with good accuracy, demonstrating its usefulness for automated intrusion detection in large-scale network environments.

PROBLEM STATEMENT :

Botnets are networks of compromised devices used by attackers to perform malicious activities such as Distributed Denial of Service (DDoS), data theft, and reconnaissance attacks. Traditional detection methods, including signature-based and rule-based approaches, fail to effectively detect modern P2P botnets due to their decentralized communication structure and dynamic behavior.

Therefore, there is a need for an intelligent system that can analyze network traffic patterns and automatically detect botnet activity. This project addresses the problem by using a CNN-based deep learning model trained on network flow features to identify botnet attacks efficiently

OVERALL ARCHITECTURE:



1.Data Preparation:

Dataset Used: The dataset used is a processed version of the BoT-IoT dataset, specifically the file: UNSW_2018_IoT_Botnet_Final_10_Best.csv This dataset contains pre-extracted network flow features representing both normal and botnet traffic

2. Feature Selection:

A subset of 10 important flow-based features is selected: Seq, stddev, N_IN_Conn_P_SrcIP ,N_IN_Conn_P_DstIP ,min ,max ,mean ,State_number ,srate ,drate

3. Data Preprocessing:

The dataset is processed as follows:

- Column names are cleaned
- Categorical values are converted into numerical format using label encoding
- Target variable is automatically selected (attack/category/subcategory)
- Irrelevant columns are removed

4. Dataset Splitting:

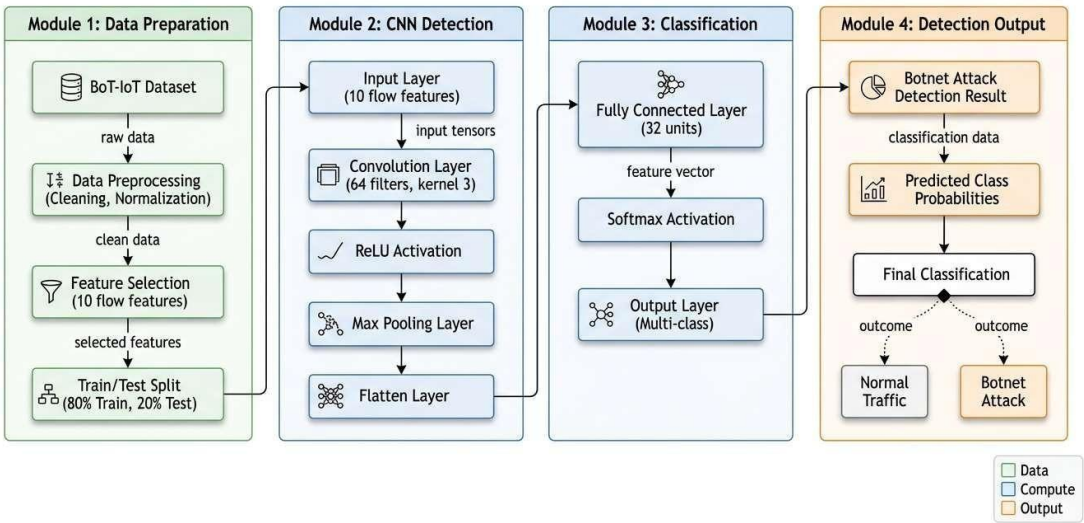
The dataset is divided into:

- 80% Training Data
- 20% Testing Data

Detection Model:

- The system uses a 1D Convolutional Neural Network (CNN) to detect botnet activity from network flow features.

- **Architecture**
- **Input Layer**
  - Receives 10 selected features.
- **Convolution Layer**
  - 64 filters.
  - Kernel size = 3.
  - Extracts feature patterns.
- **ReLU Activation**
  - Introduces nonlinearity.
- **Max Pooling Layer**
  - Reduces dimensionality.
- **Flatten Layer**
  - Converts the pooled data into a 1D vector.
- **Dense Layer**
  - 32 neurons.
  - Learns complex relationships between the extracted features.
- **Output Layer**
  - Uses Softmax activation for final classification.



Module 1: Data Preparation :

- Data loading from CSV
- Feature selection (10 features only)
- Encoding categorical values
- Train-test split
- Reshaping for CNN

Module 2: CNN Detection

- CNN extracts patterns from numerical feature vectors (not images)
- Convolution identifies relationships between flow features
- Pooling reduces feature size
- Flatten prepares data for classification

Module 3: Classification

- Dense layer learns feature combinations
- Softmax converts outputs into class probabilities
- Highest probability determines predicted class

Module 4: Detection Output

- Output = predicted class label
- Used to identify normal or botnet traffic

PSEUDOCODE / ALGORITHM FOR EACH MODULE:

MODULE 1: DATA COLLECTION

Algorithm: Data Collection

Input: UNSW IoT Botnet Dataset (CSV file)

Output: Loaded dataset

1. Import pandas library

2. Load dataset from CSV file using read_csv()

3. Display dataset shape and preview

4. Store dataset in a DataFrame

End

MODULE 2: DATA PREPROCESSING

Algorithm: Data Preprocessing

Input: Raw dataset

Output: Processed dataset (X, y) 1.

Clean column names (remove spaces)

2. Identify target column:

    - If 'attack' exists → use it

    - Else if 'category' exists → use it

    - Else if 'subcategory' exists → use it

    - Else → use last column

3. Separate features (X) and target (y) 4. Remove unnecessary columns (e.g., 'Unnamed: 0')

5. For each column in X:

    If column is categorical:

Apply Label Encoding

6. Apply Label Encoding to target variable (y)

7. Select required features:

['seq', 'stddev', 'N_IN_Conn_P_SrcIP', 'min',

'state_number', 'mean', 'N_IN_Conn_P_DstIP', 'drate', 'srate', 'max']

End

MODULE 3: DATA SPLITTING:

Algorithm: Train-Test Split

Input: Features (X), Labels (y)

Output: X_train, X_test, y_train, y_test

1. Split dataset using train_test_split()

2. Set test size = 0.2 (20%)

3. Random state can be set for reproducibility

End

MODULE 4: DATA REDUCTION:

Algorithm: Data Sampling

Input: Full dataset

Output: Reduced dataset

1. Randomly select 50,000 samples from dataset

2. Use fixed random state for consistency

3. Use sampled dataset for training

End

MODULE 5: FEATURE RESHAPING:

Algorithm: Feature Reshaping Input:

X_train, X_test

Output: Reshaped data for CN

1. Convert data into NumPy arrays

2. Reshape X_train into (samples, features, 1)

3. Reshape X_test into (samples, features, 1)

4. Verify shape compatibility with CNN End

MODULE 6: CNN MODEL BUILDIN

Algorithm: CNN Model

Input: Training data

Output: Trained CNN model 1.

Initialize Sequential model

2. Add Conv1D layer:

    - Filters = 64

    - Kernel size = 3

    - Activation = ReLU

3. Add MaxPooling1D layer

4. Add Flatten layer 5. Add Dense layer:

    - Units = 32 - Activation = ReLU

6. Add Output layer:

    - Units = number of unique classes in y

    - Activation = Softmax

7. Compile model:

    - Optimizer = Adam

- Loss = sparse categorical crossentropy
- Metrics = accuracy

End

MODULE 7: MODEL TRAINING:

Algorithm: Model Training

Input: Training dataset

Output: Trained model 1.

Train model using fit()

2. Set:
    - Epochs = 2
    - Batch size = 256

3. Store training history

End

MODULE 8: PREDICTION

Algorithm: Prediction

Input: Test dataset

Output: Predicted labels

1. Use trained model to predict probabilities

2. Convert probabilities into class labels using argmax

3. Store predicted labels

End

MODULE 9: EVALUATION:

Algorithm: Model Evaluation

Input: Actual labels (y_test), Predicted labels (y_pred)

Output: Performance metrics

1. Calculate Accuracy

2. Generate Classification Report:

    - Precision

    - Recall

    - F1-score

3. Generate Confusion Matrix

End

DATASET DESCRIPTION:

The dataset used in this project is a processed version of the BoT-IoT dataset, specifically the file UNSW_2018_IoT_Botnet_Final_10_Best.csv. It contains network traffic data with both normal and botnet attack activities generated in an IoT environment.

The dataset includes flow-based features such as sequence number, flow duration, packet rates, and connection statistics, which effectively represent network behavior. A subset of 10 important features is selected for model training.

Due to the large size of the dataset, a reduced sample of 50,000 records is used for efficient processing. The dataset is labeled, allowing the CNN model to classify traffic as normal or botnet attack.

Code:

#Loading Dataset : import pandas as pd
data=pd.read_csv("UNSW_2018_IoT_Botnet_Final_10_Best.csv",sep=';'
) print(data.head())

print(data.shape)

```
   Unnamed: 0  pkSeqID proto            saddr  sport            daddr dport  \
0           0        1   tcp  192.168.100.147  49960    192.168.100.7    80
1           1        2   arp    192.168.100.7     -1  192.168.100.147    -1
2           2        3   tcp  192.168.100.147  49962    192.168.100.7    80
3           3        4   tcp  192.168.100.147  49964    192.168.100.7    80
4           4        5   tcp  192.168.100.147  49966    192.168.100.7    80

   seq    stddev  N_IN_Conn_P_SrcIP       min  state_number      mean  \
0    9  0.068909                 75  0.000000             1  0.068909
1   10  0.000000                  2  0.000131             2  0.000131
2   11  0.064494                 75  0.000000             1  0.064494
3   12  0.064189                 75  0.000000             1  0.064189
4   13  0.063887                 75  0.000000             1  0.063887

   N_IN_Conn_P_DstIP      drate     srate       max  attack category  \
0                 96  14.511893  0.566862  0.137818       1      DoS
1                  1   0.000000  0.000000  0.000131       1      DoS
2                 96  15.505319  0.567549  0.128988       1      DoS
3                 96  15.578993  0.567570  0.128378       1      DoS
4                 96  15.652637  0.567630  0.127774       1      DoS

  subcategory
0        HTTP
1        HTTP
2        HTTP
3        HTTP
4        HTTP
(3668522, 20)
```

#selecting the features:

Features=['seq',stddev','N_IN_Conn_P_SrcIP','min','state_number','mean','N_IN_Conn_P_DstIP','drate','srate','max']

X = data[features] y

= data['attack']

print(X.head())

```
   seq    stddev  N_IN_Conn_P_SrcIP       min  state_number      mean  \
0    9  0.068909                 75  0.000000             1  0.068909
1   10  0.000000                  2  0.000131             2  0.000131
2   11  0.064494                 75  0.000000             1  0.064494
3   12  0.064189                 75  0.000000             1  0.064189
4   13  0.063887                 75  0.000000             1  0.063887

   N_IN_Conn_P_DstIP      drate     srate       max
0                 96  14.511893  0.566862  0.137818
1                  1   0.000000  0.000000  0.000131
2                 96  15.505319  0.567549  0.128988
3                 96  15.578993  0.567570  0.128378
4                 96  15.652637  0.567630  0.127774
```

h

print(data.head())

```
     Unnamed: 0  pkSeqID proto            saddr  sport            daddr dport  \
  0           0        1   tcp  192.168.100.147  49960    192.168.100.7    80
  1           1        2   arp    192.168.100.7     -1  192.168.100.147    -1
  2           2        3   tcp  192.168.100.147  49962    192.168.100.7    80
  3           3        4   tcp  192.168.100.147  49964    192.168.100.7    80
  4           4        5   tcp  192.168.100.147  49966    192.168.100.7    80

     seq    stddev  N_IN_Conn_P_SrcIP       min  state_number      mean  \
  0    9  0.068909                 75  0.000000             1  0.068909
  1   10  0.000000                  2  0.000131             2  0.000131
  2   11  0.064494                 75  0.000000             1  0.064494
  3   12  0.064189                 75  0.000000             1  0.064189
  4   13  0.063887                 75  0.000000             1  0.063887

     N_IN_Conn_P_DstIP      drate     srate       max  attack category  \
  0                 96  14.511893  0.566862  0.137818       1      DoS
  1                  1   0.000000  0.000000  0.000131       1      DoS
  2                 96  15.505319  0.567549  0.128988       1      DoS
  3                 96  15.578993  0.567570  0.128378       1      DoS
  4                 96  15.652637  0.567630  0.127774       1      DoS

    subcategory
  0        HTTP
  1        HTTP
  2        HTTP
  3        HTTP
  4        HTTP
```

print(data.columns)

```
Index(['Unnamed: 0', 'pkSeqID', 'proto', 'saddr', 'sport', 'daddr', 'dport',
       'seq', 'stddev', 'N_IN_Conn_P_SrcIP', 'min', 'state_number', 'mean',
       'N_IN_Conn_P_DstIP', 'drate', 'srate', 'max', 'attack', 'category',
       'subcategory'],
      dtype='object')
```

#Encode Data: import
pandas as pd
from sklearn.preprocessing import LabelEncoder data =
pd.read_csv(r"UNSW_2018_IoT_Botnet_Final_10_Best.csv",sep=';',
low_memory=False
)
# Clean column names
data.columns = data.columns.str.strip()
print("Columns:", data.columns) for
col in data.columns: if
data[col].dtype == 'object':
    data[col] = data[col].astype(str)
target_col = None for col in ['attack',
'category', 'subcategory']: if col in
data.columns:
    target_col = col
break if target_col is
None:
  target_col = data.columns[-1] print("Using
target column:", target_col)
# Split data
X = data.drop(columns=[target_col], errors='ignore')
y = data[target_col] if 'Unnamed: 0' in X.columns:
X = X.drop(['Unnamed: 0'], axis=1) le =
LabelEncoder() for col in X.columns: if
X[col].dtype == 'object': X[col] =
le.fit_transform(X[col]) y = le.fit_transform(y)
print(X.head()) print("Shape:", X.shape)

```
Columns: Index(['Unnamed: 0', 'pkSeqID', 'proto', 'saddr', 'sport', 'daddr', 'dport',
       'seq', 'stddev', 'N_IN_Conn_P_SrcIP', 'min', 'state_number', 'mean',
       'N_IN_Conn_P_DstIP', 'drate', 'srate', 'max', 'attack', 'category',
       'subcategory'],
      dtype='object')
Using target column: attack
   pkSeqID  proto  saddr  sport  daddr  dport  seq    stddev  \
0        1      3      1  44407     21   6650    9  0.068909
1        2      0     12      0     10      0   10  0.000000
2        3      3      1  44409     21   6650   11  0.064494
3        4      3      1  44411     21   6650   12  0.064189
4        5      3      1  44413     21   6650   13  0.063887

   N_IN_Conn_P_SrcIP       min  state_number      mean  N_IN_Conn_P_DstIP  \
0                 75  0.000000             1  0.068909                 96
1                  2  0.000131             2  0.000131                  1
2                 75  0.000000             1  0.064494                 96
3                 75  0.000000             1  0.064189                 96
4                 75  0.000000             1  0.063887                 96

        drate     srate       max  category  subcategory
0  14.511893  0.566862  0.137818         1            1
1   0.000000  0.000000  0.000131         1            1
2  15.505319  0.567549  0.128988         1            1
3  15.578993  0.567570  0.128378         1            1
4  15.652637  0.567630  0.127774         1            1
Shape: (3668522, 18)
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
print(X_train.shape, X_test.shape)
```

```
(2934817, 18) (733705, 18)
```

```
# Reshape for CNN import
numpy as np X_train =
X_train.values
X_test = X_test.values
X_train = X_train.reshape(X_train.shape[0], X_train.shape[1], 1)
X_test = X_test.reshape(X_test.shape[0], X_test.shape[1], 1)
print(X_train.shape) import numpy as np
from tensorflow.keras.models import Sequential from
tensorflow.keras.layers import Conv1D, MaxPooling1D, Flatten,
Dense, Input
num_classes = len(np.unique(y_train))
print("Number of distinct attack classes found:", num_classes)
```

```
(2934817, 18, 1)
```

CNN Model model
= Sequential()
model.add(Input(shape=(X_train.shape[1], 1)))
model.add(Conv1D(64, 3, activation='relu'))
model.add(MaxPooling1D(2)) model.add(Flatten())
model.add(Dense(32, activation='relu')) model.add(Dense(num_classes, activation='softmax'))
model.summary()

```
Number of distinct attack classes found: 2

Model: "sequential"
```

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 16, 64)	256
max_pooling1d (MaxPooling1D)	(None, 8, 64)	0
flatten (Flatten)	(None, 512)	0
dense (Dense)	(None, 32)	16,416
dense_1 (Dense)	(None, 2)	66

```
Total params: 16,738 (65.38 KB)

Trainable params: 16,738 (65.38 KB)

Non-trainable params: 0 (0.00 B)
```

Compile Model
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy']) # Train Model
history = model.fit(X_train, y_train, epochs=2, batch_size=256)

```
Epoch 1/2
11465/11465 ━━━━━━━━━━━━━━━━━━━━ 50s 4ms/step - accuracy: 0.9996 - loss: 18.4344
Epoch 2/2
11465/11465 ━━━━━━━━━━━━━━━━━━━━ 47s 4ms/step - accuracy: 0.9997 - loss: 0.4198
```

Test accuracy
loss, acc = model.evaluate(X_test, y_test) print("Accuracy:", acc)

```
22929/22929 ━━━━━━━━━━━━━━━━━━━━ 43s 2ms/step - accuracy: 0.9999 - loss: 0.0030
Accuracy: 0.9998610019683838
```

from imblearn.over_sampling import SMOTE import numpy as np
from tensorflow.keras.callbacks import EarlyStopping
print("SMOTE BEFORE:") unique, counts =

```
np.unique(y_train, return_counts=True)
print(dict(zip(unique, counts)))
X_train_2d = X_train.reshape(X_train.shape[0], X_train.shape[1]) smote
= SMOTE(random_state=42)
X_train_smote, y_train_smote = smote.fit_resample(X_train_2d, y_train)
print("\nSMOTE AFTER:") unique, counts = np.unique(y_train_smote,
return_counts=True) print(dict(zip(unique, counts)))
X_train_smote = X_train_smote.reshape(X_train_smote.shape[0],
X_train_smote.shape[1], 1) early_stop =
EarlyStopping(monitor='val_loss', patience=3,
restore_best_weights=True) history = model.fit(
    X_train_smote, y_train_smote,
epochs=5,     batch_size=512,
validation_data=(X_test, y_test),
callbacks=[early_stop]
)
```

```
SMOTE BEFORE:
{np.int64(0): np.int64(376), np.int64(1): np.int64(2934441)}

SMOTE AFTER:
{np.int64(0): np.int64(2934441), np.int64(1): np.int64(2934441)}
Epoch 1/5
11463/11463 ━━━━━━━━━━━━━━━━━━━━ 64s 6ms/step - accuracy: 0.8581 - loss: 0.4514 - val_accuracy: 0.9147 - val_loss: 0.1903
Epoch 2/5
11463/11463 ━━━━━━━━━━━━━━━━━━━━ 61s 5ms/step - accuracy: 0.9383 - loss: 0.1950 - val_accuracy: 0.9087 - val_loss: 0.2113
Epoch 3/5
11463/11463 ━━━━━━━━━━━━━━━━━━━━ 62s 5ms/step - accuracy: 0.9445 - loss: 0.2024 - val_accuracy: 0.8918 - val_loss: 0.2517
Epoch 4/5
11463/11463 ━━━━━━━━━━━━━━━━━━━━ 74s 6ms/step - accuracy: 0.9496 - loss: 0.1755 - val_accuracy: 0.9276 - val_loss: 0.1901
Epoch 5/5
11463/11463 ━━━━━━━━━━━━━━━━━━━━ 63s 5ms/step - accuracy: 0.9518 - loss: 0.1782 - val_accuracy: 0.8933 - val_loss: 0.2489
```

```
import numpy as np

From sklearn.metrics import classification_report, confusion_matrix

loss, acc = model.evaluate(X_test, y_test) print(f"CNN Test

Accuracy: {acc * 100:.2f}%\n")

# Predictions y_pred_prob =

model.predict(X_test)  y_pred =

np.argmax(y_pred_prob, axis=1)

# Final Reports print("Classification

Report:\n")
```

print(classification_report(y_test, y_pred))

print("\nConfusion Matrix:\n")

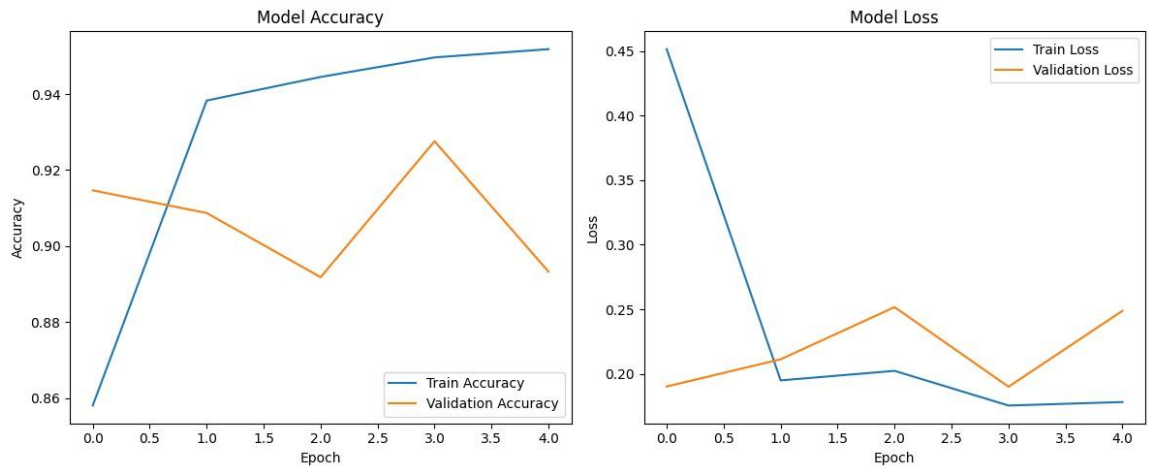
print(confusion_matrix(y_test, y_pred))

```
22929/22929 ━━━━━━━━━━━━━━━━━━━━ 46s 2ms/step - accuracy: 0.9276 - loss: 0.1901
CNN Test Accuracy: 92.76%

22929/22929 ━━━━━━━━━━━━━━━━━━━━ 33s 1ms/step
Classification Report:

              precision    recall  f1-score   support

           0       0.00      1.00      0.00       101
           1       1.00      0.93      0.96    733604

    accuracy                           0.93    733705
   macro avg       0.50      0.96      0.48    733705
weighted avg       1.00      0.93      0.96    733705


Confusion Matrix:

[[   101      0]
 [ 53130 680474]]
```

import matplotlib.pyplot as plt plt.figure(figsize=(12, 5)) plt.subplot(1, 2, 1) plt.plot(history.history['accuracy'], label='Train Accuracy') plt.plot(history.history['val_accuracy'], label='Validation Accuracy') plt.title('Model Accuracy') plt.ylabel('Accuracy') plt.xlabel('Epoch') plt.legend(loc='lower right') plt.subplot(1, 2, 2) plt.plot(history.history['loss'], label='Train Loss') plt.plot(history.history['val_loss'], label='Validation Loss') plt.title('Model Loss') plt.ylabel('Loss') plt.xlabel('Epoch')

plt.legend(loc='upper right')

plt.tight_layout() plt.show()



import matplotlib.pyplot as plt import seaborn as sns from

sklearn.metrics import confusion_matrix, roc_curve, auc

--- 1. Confusion Matrix Plot --- cm =

confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8, 6))

Heatmap sns.heatmap(cm, annot=True, fmt='d',

cmap='Blues',

xticklabels=['Normal (Class 0)', 'Botnet (Class 1)'],

yticklabels=['Normal (Class 0)', 'Botnet (Class 1)'])

plt.title('Confusion Matrix - Botnet Detection')

plt.ylabel('True Label') plt.xlabel('Predicted Label')

plt.show() print("-" * 50)

--- 2. ROC Curve Plot --- y_pred_prob_class1

= y_pred_prob[:, 1] fpr, tpr, thresholds =

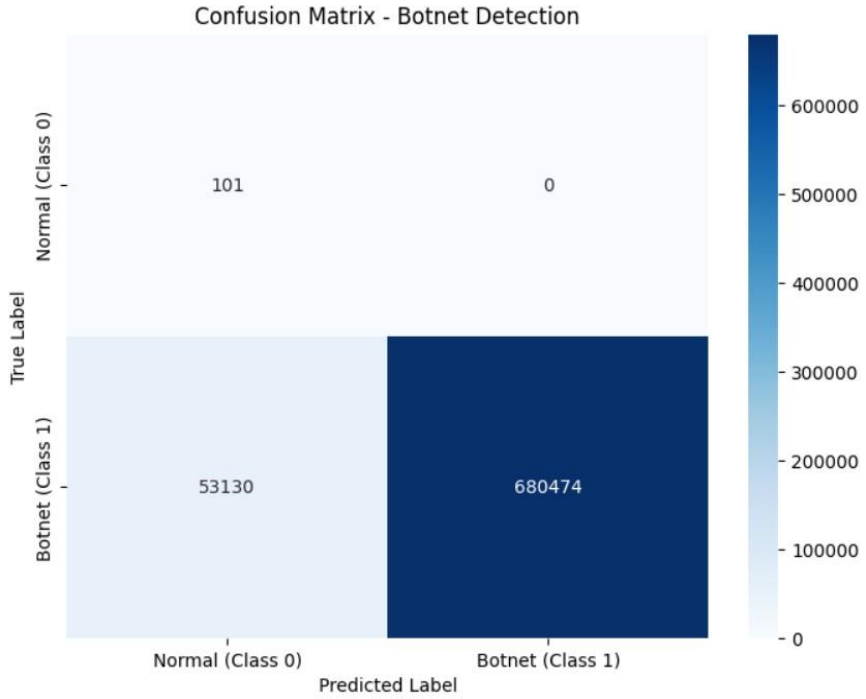
roc_curve(y_test, y_pred_prob_class1) roc_auc

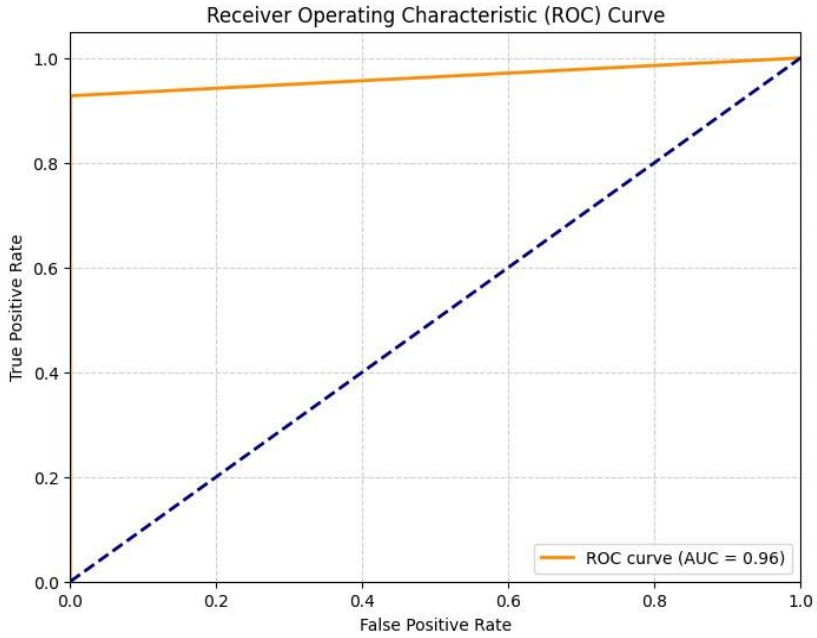
= auc(fpr, tpr) plt.figure(figsize=(8, 6))

plt.plot(fpr, tpr, color='darkorange', lw=2, label=f'ROC curve (AUC = {roc_auc:.2f})') plt.plot([0, 1], [0, 1], color='navy', lw=2, linestyle='--') #

Center diagonal line plt.xlim([0.0, 1.0]) plt.ylim([0.0, 1.05]) plt.xlabel('False

Positive Rate') plt.ylabel('True Positive Rate') plt.title('Receiver Operating Characteristic (ROC) Curve') plt.legend(loc="lower right") plt.grid(True, linestyle='--', alpha=0.6) plt.show()





RESULTS AND DISCUSSION:

The CNN model was trained using the selected network flow features from the UNSW IoT Botnet dataset. The model was evaluated on the test dataset and achieved a good level of accuracy in detecting botnet traffic.

The confusion matrix shows that the model correctly classifies most of the botnet traffic samples, while a smaller number of normal traffic instances are misclassified. This indicates that the model is more effective in identifying malicious traffic.

The classification report shows that precision, recall, and F1-score values are higher for the botnet class compared to the normal class. This is mainly due to the imbalance in the dataset, where botnet samples are more dominant.

The ROC curve demonstrates a high Area Under Curve (AUC) value, indicating that the model has a strong ability to distinguish between normal and botnet traffic.

Overall, the CNN-based approach is effective in detecting P2P botnet activities using flow-based features, though performance can be further improved with better data balancing and training.

CONCLUSION:

This project presented a CNN-based deep learning model for detecting P2P botnet attacks using network flow features. A processed subset of the BoTIoT dataset was used for training and testing the model. The system successfully classified network traffic into normal and botnet categories with good accuracy.

The results show that Convolutional Neural Networks can effectively learn patterns from network traffic data and can be used for automated intrusion detection. Despite some limitations such as reduced dataset size and class imbalance, the model demonstrates strong potential for improving network security in real-world environments.